# Identifying LLMS From Their Responses

CMPSC 448 Midterm Project Report

*Simon Freitag*

## Problem Definition and Dataset Curation

This project asks whether LLM-generated text contains enough model-specific information to identify which model produced it. The task is a four-class supervised classification problem with labels ChatGPT, Claude, Llama, and Gemini. A Convolutional Neural Network (CNN) and a Long Short-Term Memory network (LSTM) were used for the required classification tests.

### Dataset Configuration

I collected the data from a public, open-source dataset on Hugging Face called PersonalLLM. The dataset contained 8 prompt/response pairs for 8 different LLM models. I selected one model from each family as required by the project assignment: GPT-4o, Claude-3-Sonnet, Llama-3-70B-Instruct, and Gemini-Pro-1.5. Each prompt was kept only when all four models had valid responses in the dataset. That prompt then produced four examples, one for each model. This created exactly balanced classes and ensured that the same prompts were available to every label.

After curation, the training dataset retained 9,402 prompts with 37,608 examples, while the testing dataset retained 1,000 prompts with 4,000 total examples. Since all four classes are balanced, random guessing has an expected accuracy of 25%.

Text is split with whitespace tokenization. Prompts are limited to 100 words, and responses are limited to 150 words. A separate vocabulary is built from the training portion of each experiment and is limited to 20,000 entries. Padding values use ID = 0, and unknown word values use ID = 1. The original sequence length is also saved so the LSTM can later select the final non-padded position.

## CNN and LSTM Implementation and Training

### Model Design

The CNN uses a 64-value embedding for each token, which gives every word a learned numerical representation. These embeddings are then passed through a one-dimensional convolution with 64 filters and a kernel size of three. Each filter moves across the text three tokens at a time and

learns to respond strongly to certain local word patterns that may be characteristic of a particular LLM. ReLU is applied after convolution so negative values are removed, leaving the strongest positive pattern detections. Global max pooling then keeps the strongest detected pattern from each of the 64 filters across the entire response. This produces a compact representation of the most important local features found in the text, which is passed to a linear layer that produces one score for each of the four LLM classes.

The LSTM uses the same 64-value token embeddings and a hidden size of 64. Unlike the CNN, which looks for local patterns, the LSTM processes the text one token at a time in order. At each position, it combines the current token with information carried forward from earlier tokens, allowing its hidden state to represent the sequence that has been read so far. Because every example is padded to the same length, the saved sequence length is used to select the LSTM output at the final real token rather than at a padding position. This final 64-value representation summarizes the information the LSTM has retained from the response. It is then passed to a linear layer that produces four class scores, and the class with the highest score is used as the model's prediction.

### Training and Testing

Both models used the same values for the following training settings:

- Batch size = 64
- Learning rate = 0.01
- Vocabulary size = 20,000
- Embedding size = 64
- Hidden size = 64
- Input max length = 100
- Output max length = 150

The CNN model was trained for 5 epochs, while the LSTM was trained for 15 epochs. This was due to the LSTM learning at a slower pace, making more epochs necessary to get a true statistic on the performance of the model. Both models used cross-entropy and stochastic gradient descent. For each batch, gradients are cleared, predictions are generated, the loss is calculated, gradients are computed backwards, and the optimizer updates the model weights. Accuracy is reported as the number of correct predictions divided by the number of test examples.

## Results

The main experiment was run for three text settings: input only, output only, and a combination of input and output. For each setting, a vocabulary was built from its training data, both models were trained using the settings above, and final accuracy was measured on the 4,000 test examples. Results can be viewed in the table below.

| Model | Input Only | Output Only | Input + Output |
|---|---|---|---|
| CNN | 25.00% | 72.72% | 72.32% |
| LSTM | 25.00% | 36.93% | 46.77% |

The input-only losses stayed near the level expected for an uncertain four-class classifier, and both models finished at 25.00% accuracy, which is to be expected as the models should have to randomly guess with only the input being given. With output text, the CNN loss dropped from 1.0701 to 0.6456 over five epochs and reached 72.72% accuracy. The LSTM output-only loss ended at 1.3157 and reached 36.93%. For input plus output, CNN accuracy stayed nearly unchanged at 72.32%, while LSTM accuracy increased to 46.77%.

The full output of the code can be viewed on the last page of this report.

## In-Depth Analysis and Research Questions

### RQ1: Can we identify which LLM generated a response?

Yes. Using only the generated response, both models performed above the 25% baseline. The CNN reached 72.72% accuracy, and the LSTM reached 36.93%. The gap in accuracy suggests that local short patterns are more useful for this dataset, although the LSTM result also shows that model identity can be learned from sequential response information.

### RQ2: Does the user prompt help identify the LLM?

The prompt by itself does not identify the LLM. Both models achieve exactly 25.00% because every retained prompt appears with all four labels. Additionally, adding the prompt does not improve the CNN, which went from 72.72% accuracy to a very similar 72.32% accuracy. The LSTM, however, improved from 36.93% to 46.77% accuracy when the prompt is added. This suggests that the sequential model benefits from seeing the prompt as context for the response, even though the prompt itself contains no model identity information. Conceptually, this makes sense as the prompt could, for example, ask for an output that is exactly one sentence. Then, if the sequential model understands this context, it can better identify responses that break this rule and find which models tend to exhibit this behavior more often.

## RQ3: Do LLM fingerprints generalize across tasks or domains?

The PersonalLLM subset field was used as a practical source grouping. The hh-rlhf-helpful-base subset was removed completely from training, leaving 20,248 training examples. Testing was then limited to 1,828 examples from that subset, using output text only. Accuracy results can be seen in the table below.

| Model | Normal Output | Unseen Subset | Change |
|---|---|---|---|
| CNN | 72.72% | 71.72% | -1.00% |
| LSTM | 36.93% | 31.51% | -5.42% |

The CNN lost only 1% accuracy, which suggests that most of its learned signal transfers to the unseen subset. The LSTM drops 5.42%, indicating a greater sensitivity to a shift in seen content. This experiment is not a perfectly controlled comparison because the normal results use the full mixed test set while the subset result uses only one subset, but it still provides evidence that the CNN fingerprint is more generalized than the LSTM fingerprint.

## RQ4: What characteristics distinguish different LLMS?

Several response characteristics differ across the four model families. The statistics below use the same first 150 output words visible to the classifiers.

| LLM | Avg. Words | Unique Vocabulary | Lexical Diversity | Avg. Formatting Tokens |
|---|---|---|---|---|
| ChatGPT | 135.97 | 30,177 | 0.7343 | 7.70 |
| Claude | 139.23 | 28,259 | 0.7098 | 4.38 |
| Llama | 141.81 | 29,646 | 0.6908 | 5.32 |
| Gemini | 134.11 | 33,102 | 0.7530 | 11.98 |

Gemini has the largest unique vocabulary, highest lexical diversity, and the most formatting tokens from this dataset. Llama has the longest average responses and the lowest lexical diversity. Claude uses the fewest formatting tokens. These differences show that the models vary in both language use and presentation style.

To test whether formatting contributes to classification, common Markdown symbols such as asterisks, number signs, greater-than signs, and hyphens were removed from both training and

test outputs. The models were then retrained using the same output-only setup. The results can be seen in the table below.

| Model | Original Output | Formatting Removed | Change |
|---|---|---|---|
| CNN | 72.72% | 64.28% | -8.44% |
| LSTM | 36.93% | 32.12% | -4.81% |

Formatting removal reduces the CNN accuracy by 8.44% and the LSTM accuracy by 4.81%. This shows that structural formatting is useful for both models, especially the CNN. However, both models still remain above the baseline of 25.00% for random guessing, so linguistic patterns also provably contribute to classification.

## Lessons and Experience

A major early challenge was turning the original dataset into a balanced classification problem. Not every prompt contained all four target models, so keeping incomplete prompt groups would have changed the output significantly and given bad data. Requiring all four responses fixed that issue and also made the 25.00% input-only output much simpler to explain.

Similarly, the biggest overall implementation challenge was converting the PersonalLLM dataset into a form that both networks could use correctly. Each original prompt could contain responses from up to eight different models, so the code first had to identify the four selected LLM families, match each response to the correct label, and discard prompts that did not contain all four responses. After that, the text had to be converted into fixed-length numerical sequences for batching. This required building a vocabulary, handling unknown words, adding padding, and keeping track of the true sequence length so the LSTM would use the final real token rather than a padding position. The CNN also required the embedding tensor dimensions to be rearranged before applying Conv1d. Working through these details showed how much of a neural network implementation depends on preparing the data in exactly the format expected by each layer.

The project helped solidify the text-processing process in my mind. Tokenization, vocabulary IDs, padding, embeddings, and saved sequence lengths all had to work and chain together before either of the networks would train correctly. The LSTM was especially sensitive to sequence handling since its final representation had to come from the last real token rather than padded positions.

Training also showed that two architectures can behave very differently under the same data. The CNN learned quickly and performed well after five epochs, while the LSTM showed much slower improvements and required more training, even while still yielding a weaker result. Response

questions three and four also showed why accuracy alone is not enough for analysis. The tests done for these questions tested whether the learned patterns transfer to unseen domains, while removing formatting tokens showed that part of the CNN's advantage comes from structural presentation rather than the linguistic patterns alone. These experiments taught that there is much more beneath the surface to be considered when it comes to these trained models.

## Conclusion

The results show that generated responses contain measurable signals that can identify their source LLM. The CNN achieved 72.72% accuracy using output text alone, far above the 25% random baseline, while the LSTM reached 36.93%. The prompt alone was not enough for the networks to predict the LLM, however, adding it improved the LSTM network. The cross-subset test showed stronger CNN generalization, and the formatting removal experiment showed that formatting is an important but incomplete part of the learned fingerprint. Overall, the four model families differ in both linguistic and structural characteristics that can be detected with neural classifiers.

## Complete Output

Training examples: 37608 | Testing examples: 4000
Using device: cpu

input | vocabulary size = 20000
CNN
  Epoch 1: loss = 1.4009
  Epoch 2: loss = 1.3965
  Epoch 3: loss = 1.3945
  Epoch 4: loss = 1.3929
  Epoch 5: loss = 1.3917
  Test accuracy = 0.2500
LSTM
  Epoch 1: loss = 1.3889
  Epoch 2: loss = 1.3880
  Epoch 3: loss = 1.3878
  Epoch 4: loss = 1.3877
  Epoch 5: loss = 1.3876
  Epoch 6: loss = 1.3875
  Epoch 7: loss = 1.3874
  Epoch 8: loss = 1.3874
  Epoch 9: loss = 1.3873
  Epoch 10: loss = 1.3873
  Epoch 11: loss = 1.3872
  Epoch 12: loss = 1.3872
  Epoch 13: loss = 1.3871
  Epoch 14: loss = 1.3871
  Epoch 15: loss = 1.3870
  Test accuracy = 0.2500

output | vocabulary size = 20000
CNN
  Epoch 1: loss = 1.0701
  Epoch 2: loss = 0.7890
  Epoch 3: loss = 0.7174
  Epoch 4: loss = 0.6778
  Epoch 5: loss = 0.6456
  Test accuracy = 0.7272
LSTM
  Epoch 1: loss = 1.3868
  Epoch 2: loss = 1.3819
  Epoch 3: loss = 1.3788
  Epoch 4: loss = 1.3757
  Epoch 5: loss = 1.3727
  Epoch 6: loss = 1.3695

```
  Epoch 7: loss = 1.3659
  Epoch 8: loss = 1.3619
  Epoch 9: loss = 1.3573
  Epoch 10: loss = 1.3518
  Epoch 11: loss = 1.3445
  Epoch 12: loss = 1.3334
  Epoch 13: loss = 1.3068
  Epoch 14: loss = 1.3588
  Epoch 15: loss = 1.3157
  Test accuracy = 0.3693

input_output | vocabulary size = 20000
CNN
  Epoch 1: loss = 1.0754
  Epoch 2: loss = 0.8047
  Epoch 3: loss = 0.7339
  Epoch 4: loss = 0.6912
  Epoch 5: loss = 0.6545
  Test accuracy = 0.7232
LSTM
  Epoch 1: loss = 1.3866
  Epoch 2: loss = 1.3815
  Epoch 3: loss = 1.3784
  Epoch 4: loss = 1.3755
  Epoch 5: loss = 1.3727
  Epoch 6: loss = 1.3696
  Epoch 7: loss = 1.3664
  Epoch 8: loss = 1.3628
  Epoch 9: loss = 1.3585
  Epoch 10: loss = 1.3531
  Epoch 11: loss = 1.3459
  Epoch 12: loss = 1.3343
  Epoch 13: loss = 1.3054
  Epoch 14: loss = 1.2140
  Epoch 15: loss = 1.1915
  Test accuracy = 0.4677

Final Results
CNN  | input       | 0.2500
LSTM | input       | 0.2500
CNN  | output      | 0.7272
LSTM | output      | 0.3693
```

CNN  | input_output | 0.7232
LSTM | input_output | 0.4677

RQ3 - Generalization Across Subsets
Kept subset: hh-rlhf-helpful-base
Training examples without kept subset: 20248
Testing examples from kept subset: 1828

CNN
Epoch 1: loss = 1.2998
Epoch 2: loss = 1.0441
Epoch 3: loss = 0.9601
Epoch 4: loss = 0.9017
Epoch 5: loss = 0.8563

LSTM
Epoch 1: loss = 1.3892
Epoch 2: loss = 1.3847
Epoch 3: loss = 1.3818
Epoch 4: loss = 1.3795
Epoch 5: loss = 1.3776
Epoch 6: loss = 1.3759
Epoch 7: loss = 1.3743
Epoch 8: loss = 1.3727
Epoch 9: loss = 1.3711
Epoch 10: loss = 1.3695
Epoch 11: loss = 1.3678
Epoch 12: loss = 1.3659
Epoch 13: loss = 1.3640
Epoch 14: loss = 1.3621
Epoch 15: loss = 1.3599

RQ3 Comparison
CNN normal output accuracy: 0.7272
CNN unseen-subset accuracy: 0.7172
LSTM normal output accuracy: 0.3693
LSTM unseen-subset accuracy: 0.3151

RQ4 - Response Characteristics

GPT
Average words: 135.97

Vocabulary size: 30177
Lexical diversity: 0.7343
Average formatting tokens: 7.70

Claude
Average words: 139.23
Vocabulary size: 28259
Lexical diversity: 0.7098
Average formatting tokens: 4.38

Llama
Average words: 141.81
Vocabulary size: 29646
Lexical diversity: 0.6908
Average formatting tokens: 5.32

Gemini
Average words: 134.11
Vocabulary size: 33102
Lexical diversity: 0.7530
Average formatting tokens: 11.98

RQ4 - Formatting Ablation

CNN
Epoch 1: loss = 1.3264
Epoch 2: loss = 1.0858
Epoch 3: loss = 0.9752
Epoch 4: loss = 0.9060
Epoch 5: loss = 0.8510

LSTM
Epoch 1: loss = 1.3873
Epoch 2: loss = 1.3830
Epoch 3: loss = 1.3806
Epoch 4: loss = 1.3790
Epoch 5: loss = 1.3777
Epoch 6: loss = 1.3764
Epoch 7: loss = 1.3752
Epoch 8: loss = 1.3740
Epoch 9: loss = 1.3728
Epoch 10: loss = 1.3716

Epoch 11: loss = 1.3704
Epoch 12: loss = 1.3690
Epoch 13: loss = 1.3677
Epoch 14: loss = 1.3662
Epoch 15: loss = 1.3647

RQ4 Comparison
CNN original output accuracy: 0.7272
CNN without formatting: 0.6428
LSTM original output accuracy: 0.3693
LSTM without formatting: 0.3212